

한대장 · 기술 고문 요청서 · 2026.09.03

학교 장학 시스템에 신청을 밀어 넣는 방법

대학생 장학금 앱 **한대장**은 공고 수집·자격 매칭·서류 준비·**공고 원본과 동일한 구조의 신청서 생성**까지 동작합니다. 남은 것은 마지막 한 걸음 — 학교 포털에 실제로 제출하는 일입니다. 이 문서는 그 한 걸음을 설계하기 위해 **앱의 구조·코드·데이터를 전부 공개**하고, 개발 관점에서 답이 필요한 **열두 가지**를 정리한 것입니다 — 앞의 다섯은 **학교와 있는 방법**, 뒤의 다섯은 **지금 만들어 둔 구조가 그 무게를 견디는가**입니다.

이 문서가 요청하는 것

1. **연동 아키텍처 선택** — 상시 서버가 없는 정적 PWA 구조에서 실현 가능한 방식은 무엇인가
2. **인증·세션 설계** — 학교 계정 로그인 벽을 넘는 표준 설계와, 대안의 유지비
3. **신청 데이터 계약과 역등성** — 재전송·중복·부분 실패·접수 확인을 어떻게 설계하나
4. **현 구조의 기술 리스크 리뷰** — 이대로 학교와 붙었을 때 터질 곳
5. **90일 실행 계획** — 협약 없이 지금 작동시킬 수 있는 것의 순서와 최소 구현

그리고 지금의 구조에 대하여

6. **회원 데이터 모델** — 프로필·신청내역을 jsonb 한 칸에 통째로 넣어도 되는가
7. **확장성** — 사용자가 늘면 가장 먼저 터지는 곳은 어디인가
8. **SDK 없이 직접 호출** — 보안 정책을 지키려 택한 방식의 대가는 무엇인가
9. **수집 범위** — 외부 공고를 전부 수집하는 것이 맞는가
10. **수집 차단** — 사이트마다 막는 방식이 다른데 유형별로 어떻게 접근하는가
11. **항목 추출** — 표현이 제각각인 공고에서 ‘지원 자격’만 뽑는 표준 방법이 있는가
12. **개발 방식** — AI와 대화하며 만든 코드의 한계선은 어디인가

17,050

앱 코어 코드 줄 수
빌드 도구 없음

48

스키마화된 신청서 양식
602개 입력 필드

121

채널을 실측한 공고
경희 58 + 외대 63

0

학교 시스템으로
실제 전송된 신청

작성

이선주 (개발자 ·
seonju5543@gmail.com)

대상 서비스

한대장 (한국 대학교 장학금)
모바일 PWA · 무료

저장소

github.com/seonju5543-
web/hanggonggan
공개 · 전체 열람 가능

근거 데이터

2026-09-02 경희대·한국외대
장학 공고 121건 전수 조사

00 목차

01 30초 요약 — 무엇이 되고, 무엇이 안 되는가

02 앱의 구조 — 답을 주시려면 알아야 할 전부

시스템 구성 · 스택 · 코드 지도 · 데이터 파이프라인 · 데이터 모델 · 학생 프로필 33값 · 저장/보안 경계 · 현재 신청 플로우 · 미배포 자산

03 문제 — 실측 121건이 말하는 것

채널 분포 · 포털형 18건 · 구현 후보 5안 포지셔닝 · 우리 추정치

04 질문 다섯 — 학교 연동

Q1 연동 아키텍처 · Q2 인증·세션 · Q3 데이터 계약과 역등성 · Q4 구조 리스크 리뷰 · Q5 90일 실행 계획

05 질문 일곱 — 지금의 구조

Q6 회원 데이터 모델 · Q7 확장성 · Q8 SDK 없이 직접 호출 · Q9 수집 범위 · Q10 수집 차단 · Q11 항목 추출 · Q12 개발 방식

06 부록

A 답변 후 우리가 정할 것 · B 이 문서 밖의 질문 · C 파일 트리 · D 용어 · E 접근 방법

그림·표 목록

그림 1	사용자 여정과 굵긴 지점	표 1	실행 환경과 비용
그림 2	시스템 구성도	표 2	코드 지도 (27개 파일 · 17,050줄)
그림 3	데이터 파이프라인 (게시판 → 앱)	표 3	자동화 로봇 27종
그림 4	개인정보 경계 — 기기와 서버	표 4	학생 프로필 33값
그림 5	지금 신청 플로우 vs 목표	표 5	포털형 18건의 경로
그림 6	신청 채널 분포 (121건)	표 6	구현 후보 5안 · 우리 추정
그림 7	구현 후보 포지셔닝 맵	표 7	답변 후 결정 사항
그림 8	신청 상태 머신과 증거	표 8	고문 범위 밖 질문

01 30초 요약

앱은 학생이 신청서를 **작성 완료**하는 지점까지 완성돼 있습니다. 학교 시스템에 **제출**하는 마지막 단계만 사람이 손으로 합니다. 이 문서는 그 단계를 자동화하기 위한 기술 상담 요청입니다.

17,050

앱 코어 코드 줄 수
(27개 파일 · 빌드 도구 없음)

48

스키마화된 신청서 양식
(141섹션 · 602필드)

33

앱이 이미 보유한
학생 프로필 값

0

학교 시스템으로
실제 전송된 신청

되는 것

- 공고 자동 수집 — 학교 게시판을 매일 훑는 로봇 27종(GitHub Actions). 첨부 HWP까지 받아 글자를 뽑습니다.
- 자격 매칭 — 공고 원문의 자격 요건 문장을 읽어 학생 프로필과 대조하고, **모르면 모른다고 말합니다**(추론 금지가 서비스 원칙입니다).
- 양식 작성 — 공고에 첨부된 HWP 신청서를 스키마로 옮겨(48종), 앱에서 질문에 답하면 **원본과 같은 구조의 문서**를 만듭니다.
- 서류 보관함 · 마감 알림(웹 푸시 가동 중) · AI 자기소개서 초안(서버 대기).

안 되는 것 — 그리고 그것이 전부입니다

- **제출.** 학교 포털은 학번·비밀번호 로그인 뒤에 있고, 앱은 그 문을 넘지 못합니다. 지금 앱이 하는 말은 “**이 시스템의 이 경로로 가세요**”까지입니다(공고 원문 문장을 그대로 보여 줍니다). **대신 눌러 주는 것은 못 합니다.**
- 그래서 앱은 “**신청 완료**”라고 **쓰지 않습니다**. “신청 준비 완료”라고만 씁니다 — 실제 접수가 일어나지 않은 것을 완료라고 표기하지 않는 것이 이 서비스의 운영 원칙 1번입니다.

고문께 드리는 요청의 성격

법률 자문이나 학교 행정 절차가 아니라 **구현 판단**을 여쭙습니다. 저희는 이미 다섯 가지 구현안(딥링크 자동채움 · 이메일 자동 발송 · 파일 배치 · 연계 API · 학생 계정 대행)을 놓고 비교했고, **표 6**에 각 안의 우리 쪽 추정치를 채워 두었습니다. 비어 있는 칸이 여쭙는 부분입니다. 저장소는 공개이므로 코드를 직접 보셔도 됩니다.



그림 1 사용자 여정과 끊긴 지점. 앞의 다섯 단계는 완성돼 배포 중이고, 여섯 번째에서 학교 시스템의 로그인 벽을 만납니다. 일곱 번째(결과)는 학교 전산을 볼 수 없으므로 학생이 직접 기록합니다.

02 앱의 구조

이 절은 **고문께서 답을 주시기 위해 필요한 정보 전부**입니다. 구조 · 스택 · 코드 지도 · 데이터 모델 · 저장 경계 · 현재 신청 플로우 · 아직 배포하지 않은 자산 순서로 적었습니다.

2.1 한 문장 정의

대학생이 프로필을 한 번 등록하면 교내·교외 장학금을 자동 매칭받고, 증명서류 준비와 **공고 원본 양식 작성**까지 앱에서 끝내는 모바일 PWA입니다. 공고 검색·추천·알림은 학생에게 영구 무료이며, 수익은 AI 초안 크레딧과 B2B 광고로 설계돼 있습니다.

2.2 시스템 구성



● = 운영 중 · ○ = 코드는 있으나 미배포 · 점선 = 아직 없는 연결

상시 구동 서버(VM·컨테이너)는 한 대도 없습니다. 전부 정적 파일과 이벤트 기반 함수입니다.

그림 2 시스템 구성도. 학생 기기 안에서 거의 모든 판단이 일어나고, 서버는 “발송”과 “동기화”만 합니다. 이 비대칭이 Q1·Q2·Q3의 전제입니다.

2.3 실행 환경

표 1 - 실행 환경과 비용

층	무엇	현재 상태	비용
클라이언트	정적 PWA (HTML/CSS/바닐라 JS)	빌드·번들러·프레임워크 없음. 파일을 그대로 배포	0원
호스팅	GitHub Pages (main 브랜치)	공개 저장소여야 무료. 비공개 전환 시 즉시 404	0원
자동화	GitHub Actions 27 워크플로	공개 저장소라 실행 시간 무제한 무료 . 사용량은 월 약 700분	0원
서버리스	Cloudflare Workers ×4	푸시 1종 가동, 3종 미배포	0원 (무료 한도 내)
인증·동기화	Supabase (fetch 4개 엔드포인트 만)	SDK 미사용 — CSP script-src 'self' 준수 목적	0원
AI	Claude API (초안 · 스키마화)	서류 1건 약 47원. 무료 규칙 경로 우선, 실패분 만 API	사용량 과금
관리자 화면	Cloudflare Pages + Access	가동 중 · 이메일 일회용 코드 잠금	0원
합계	고정비 0원 구조. 상시 서버 없음		≈ 0원

위 두 줄에서 “**공개 저장소**”가 두 번 나오는 것이 우연이 아닙니다 — 배포도 로봇 실행도 저장소가 공개라서 공짜입니다. 비공개로 돌리면 앱이 즉시 404가 되고, 로봇의 실행 시간에도 월 2,000분 상한이 생깁니다(현재 700분). 즉 **지금은 로봇을 늘려도 예산이 걸리지 않습니다** — 질문 09가 비용이 아니라 정확도를 묻는 이유입니다.

이 “고정비 0원 · 상시 서버 없음”이 설계 제약의 핵심입니다. 학교와의 연동이 **항상 켜져 있는 수신 서버**를 요구한다면 구조가 처음으로 바뀝니다 — Q1과 Q3에서 이 점을 여쭙습니다.

2.4 코드 지도

표 2 — 앱 코어 27개 파일 · 17,050줄 (2026-09-23 실측)

파일	줄	역할 · 연동 관점에서의 의미
app.js	3,507	화면·신청 플로우·서류 보관함·진척도 트래커. 제출 버튼이 있는 곳
style.css	3,134	화면 스타일
match-engine.js	1,203	자격 매칭 12신호 + 적합도. 화면과 서비스워커가 같은 파일 을 씀
chat.js	1,074	장학금 도우미 — 앱이 아는 것만 답함
essay.js	814	자기소개서 초안·수정·제출 전 점검
notify.js	721	알림 UI·권한·발송
forms.js	551	양식 엔진 — 스키마 → 질문 → 원본 동일 문서(.doc)
index.html	500	단일 화면 · 5단계 온보딩 · CSP 메타
data.js	456	대학·학과·서류 슬롯 사전 + 제출 채널 판정
parse-requirements.js	445	공고 원문에서 자격 요건 줄 추출
parse-amount.js	416	금액 해석 · 중복 수혜 판정 · 합산 규칙
notify-rules.js	355	알림 규칙(순수 로직) — 화면·서비스워커 공용
essay-submit-check.js	318	제출 전 점검표 — 빈칸·블라인드 유출 차단
supabase-client.js	312	서버 통신 단일 창구 + 민감정보 제거
form-plan.js	299	원본 스키마를 건드리지 않고 질문만 재구성(자동 채움 22키)
sw.js	289	서비스워커 — 네트워크 우선 캐시 · 푸시 수신
section-head.js	182	공고의 절 경계 판정(자격/제외/선발) — 3개 도구 공용
essay-quality.js	101	초안 품질 검사 — 브라우저·서버·검사 공용

27개 파일 17,050 이 밖에 수집 로봇 56개(.mjs)와 검증 드라이버 54개가 있습니다

설계 규칙 하나를 미리 말씀드립니다 — **판정 규칙은 절대 복사하지 않습니다**. 화면·서비스워커·감사 도구가 같은 파일을 불러 씁니다. 규칙이 두 벌이 되면 “화면에서 숨긴 공고를 알림이 알리는” 모순이 실제로 발생했기 때문입니다.

2.5 데이터 파이프라인



사람이 개입하는 지점 — 애매한 공고는 자동 등록하지 않고 ‘컨펌 대기’로 리포트에만 올립니다. 개발자가 직접 판단합니다.

되돌리기 — 감사가 실패하면 그 실행이 새로 등록된 공고만 골라 되돌리고 이슈를 남깁니다. 잘못된 공고가 앱에 하루도 못 나갑니다.

배포 동기화 — 로봇은 기본 브랜치에만 커밋하고, 별도 로봇이 main으로 병합해 배포합니다(읽는 곳과 쓰는 곳을 가르면 중복 수집이 발생).

데이터 총량 5.9 MB — 앱이 실제로 내려받는 것은 공고 목록 34 KB · 등록 공고 199 KB · 양식 178 KB입니다. 변경이 없으면 304로 거의 0바이트.

그림 3 게시판에서 앱까지. 전 과정이 무인이며, 감사 관문이 데이터 품질의 마지막 방어선입니다.

표 3 - 자동화 로봇 27종 (대표)

로봇	주기	하는 일
collect-scholarships	매일 2회	게시판 수집 · 공고 발행 · 리포트 이슈
browser-collect	매일 2회	진짜 Chromium으로 봇 차단·동적 게시판 수집
deep-fetch	지시형	공고 본문 전문 + 첨부 원본 확보, HWP 미리보기 텍스트 추출
link-hunter	매일	원문 주소를 못 찾은 공고를 실제로 눌러 주소를 받아 적음
kosaf-fetch	주 2회	한국장학재단 재단 목록 갱신(마감 전만 추림)
deploy-sync	수집 직후·하루 2회	기본 브랜치 → main 자동 병합(= 배포)
verify-ui	푸시마다	브라우저 드라이버 13종 전부 실행(관문)
audit-coverage	매일	등록 데이터 전수를 현재 규칙으로 재검사
push-check / push-health	수동·매일	푸시 서버 상태 및 실제 수신 시험 발송
admin-lock-check	매일	관리자 화면이 잠겨 있는지 로그인 없이 확인

2.6 데이터 모델

연동 시 학교로 넘어갈 후보가 되는 두 스키마입니다. 실제 파일에서 그대로 가져왔습니다(긴 문자열만 줄임).

data/registered.json - 정식 등록 공고 72건 · 199 KB

```
{
  "id": "reg-hufs-alumni",
  "name": "총동문회 장학금 (한국외대, 2026-2학기)",
  "type": "교외", // 교내/교외 - 접수처와 무관한 분류
  "provider": "한국외대 총동문회",
  "amount": "150만원", "amountValue": 1500000,
  "amountSpec": { "kind": "fixed", "value": 1500000, "ratio": 0 },
  "deadline": "2026-08-07", "deadlineFrom": "공고 원문",
  "eligibility": { "selective": true, "schoolOnly": "한국외국어대학교" },
  "eligibilityLines": ["가 . 선발기준", "1) 2026 년 2 학기 재학생 (휴학예정자 지원불가)"],
  "documents": ["총동문회 장학금 신청서 (앱에서 원본 양식 작성)",
    "개인정보 수집·이용 동의서 (원본 첨부 HWP 다운로드)"],
  "attachments": [{ "name": "2026-2학기 총동문회 장학금+신청서.hwp", "url": "https://dep.hufs..." }],
  "sourceUrl": "https://dep.hufs.ac.kr/bbs/student/2431/259908/artclView.do",
  "sourceKind": "official",
  "formId": "hufs-alumni-apply", // ← 아래 forms.json 의 양식과 연결
  "excerpts": ["문의처 : 서울캠퍼스 장학팀 (02-2173-2137) ..."],
  "excerptNote": "공고 원문에서 그대로 발췌 (자동)",
  "listedAt": "2026-07-16"
}
```

이 한 건이 파이프라인 전체를 보여 줍니다. 그리고 이 공고가 바로 포털형 18건 중 하나입니다 — 원문의 신청 방법이 “온라인신청(HUFS Ability) → 로그인 → 교과/비교과 → 장학신청 목록”입니다. 앱은 신청서를 만들어 줄 수 있지만 그 목록에는 닿지 못합니다.

data/forms.json - 양식 스키마 48종 · 141섹션 · 602필드 · 178 KB

```
{
  "title": "대학생 청소년 AI 교육지원장학금 활동계획서 외 3종",
  "docName": "AI교육지원장학금_활동계획서",
  "org": "한국장학재단 귀하",
  "pledge": "본인은 위 내용이 사실임을 확인하며 ...", // 서약 문구 47/48중 보유
  "sections": [
    { "heading": "1. 기본정보",
      "info": [ ["대학명", "school"], ["성명", "name"], ["학번", "studentId"] ] }, // 프로필에서 자동 채움
    { "heading": "2. 멘토링 활동 계획",
      "fields": [
        { "id": "level", "label": "희망 대상 학교급", "type": "checks",
          "q": "어느 학교급 학생을 가르치고 싶나요?", "options": ["초등학교", "중학교", "고등학교"] },
        { "id": "period", "label": "희망 활동기간", "type": "text",
          "placeholder": "예: 2026. 09. 01 ~ 2026. 12. 20" },
        { "id": "schedule", "label": "희망 스케줄", "type": "schedule" }
      ] }
    ]
  }
}
```

표 · 602개 입력 필드의 타입 분포

타입	개수	설명 · 자동 전송 가능성
text	412	한 줄 입력 — 구조화 전송에 가장 적합
textarea	102	서술형(자기소개·계획) — 길이 제한 협의 필요
checks	69	다중 선택 — 코드값 매핑 필요
checks+text	8	선택 + 기타 입력
static	6	고정 문구(안내·서약)
schedule	4	요일×시간 표 — 구조가 특수
choice	1	단일 선택
합계	602	85%가 text·textarea. 표 형태는 4건뿐

2.7 앱이 이미 보유한 학생 데이터

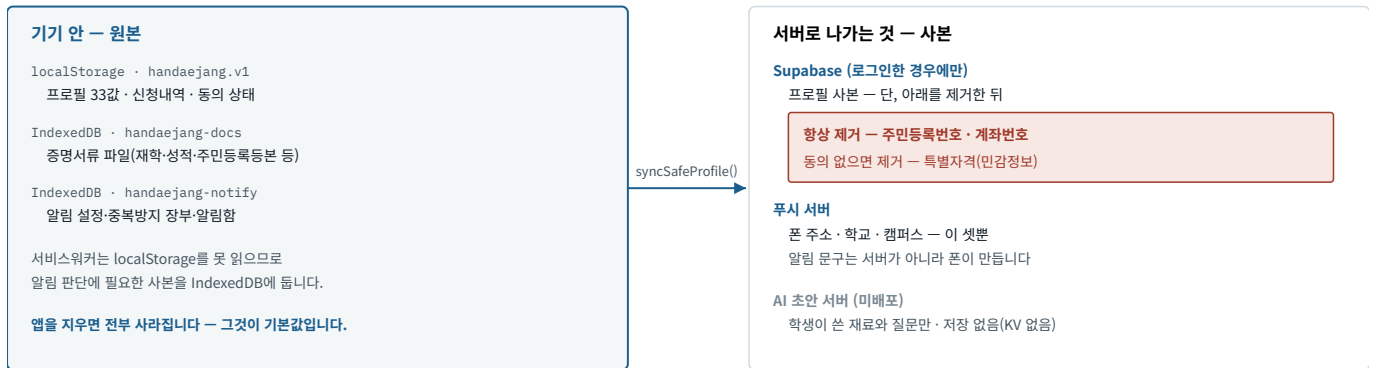
포털 자동 입력·연계 전송을 설계할 때 **출발점이 되는 값**입니다. 온보딩 5단계에서 받고, 신청서를 채우며 배운 값은 “다음에도 쓸게요”로 누적됩니다.

표 4 - 학생 프로필 33값 (실제 코드 COLLECTPROFILE() 기준)

묶음	필드	비고
신원	name · studentId · birth · birthYear · nationality · gender	gender는 신청서에서 학습
학적	school · campus · major · track · year · status · credits · gpa	매칭 12신호의 핵심
연락	phone · email · addr · emergency	addr·emergency는 학습값
보호자	guardianName · guardianRel · guardianPhone	학습값
경제	bracket (소득분위) · bank · account	계좌는 서버로 안 보냄
지역	region · regionCity · parentRegion · parentRegionCity	지역 장학금 자격
특별자격	flags[] (기초생활수급·장애·국가유공자 등)	민감정보 — 별도 동의 시에만 서버
중복 수혜	scholarships[] null	빈 배열(안 받음)과 null(모름)을 구분
기타	cert · exchange · rrn	주민번호는 서버로 안 보냄

설계상 중요한 구분 — 값이 없으면 null로 두고 **판정하지 않습니다**. 0이나 기본값을 넣으면 엉뚱한 자격 미달 판정이 납니다. 연동 스키마를 만들 때도 “모름”을 표현할 수 있어야 합니다.

2.8 저장·개인정보 경계



이 그림의 문장은 검증 드라이버가 코드로 증명합니다 — 가짜 서버를 세워 나가는 요청 본문을 전부 모아, 주민번호·계좌가 한 번도 나가지 않는지 셉니다(30행목).

그림 4 개인정보 경계. 원본은 기기에, 서버에는 걸러진 사본만 갑니다. 학교 연동은 이 경계를 처음으로 바깥으로 넓히는 일이라 Q3·Q4에서 다시 다룹니다.

supabase-client.js — 서버로 나가는 사본을 만드는 유일한 함수

```
const SYNC_OMIT_COMMON    = ['rrn', 'account']; // 항상 제거
const SYNC_SENSITIVE_KEYS = ['flags'];          // 동의했을 때만 전송

function syncSafeProfile(profile, sensitiveOk) {
  if (!profile) return null;
  const p = JSON.parse(JSON.stringify(profile));
  if (p.common) for (const k of SYNC_OMIT_COMMON) delete p.common[k];
  if (!sensitiveOk) for (const k of SYNC_SENSITIVE_KEYS) delete p[k];
  return p;
}
```

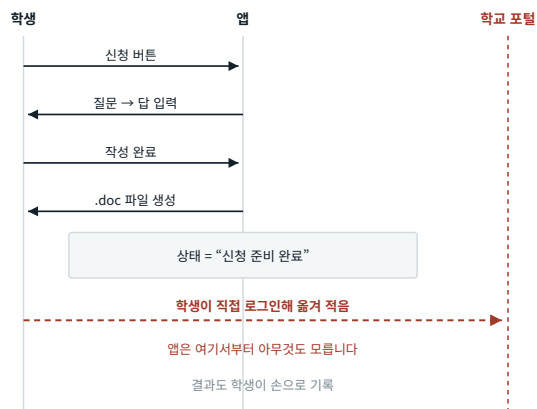
index.html — 콘텐츠 보안 정책(CSP)

```
default-src 'self'; script-src 'self';
style-src 'self' 'unsafe-inline' https://cdn.jsdelivr.net;
connect-src 'self' https://*.workers.dev https://*.supabase.co;
object-src 'none'; base-uri 'self'
```

외부 스크립트를 실행할 수 없습니다. 수집한 공고 데이터가 오염되더라도 코드 실행으로 이어지지 않게 하려는 조치이고, 동시에 **SDK를 쓸 수 없다는 제약**이기도 합니다(Supabase도 fetch로 직접 호출). 학교 연동 SDK가 필요하다면 이 정책을 어떻게 다룰지가 Q2의 일부입니다.

2.9 지금 ‘신청’ 버튼이 실제로 하는 일

지금 (배포 중)



목표 (이 문서가 설계하려는 것)

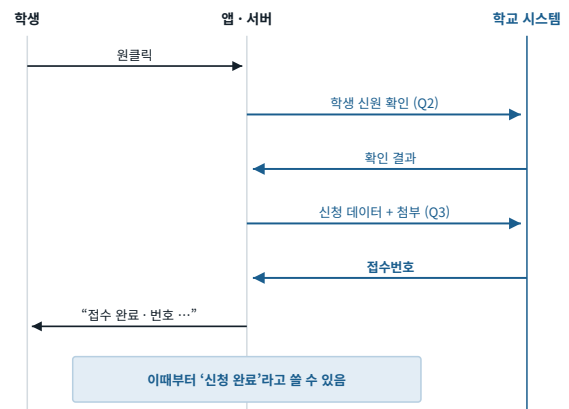


그림 5 현재 플로우와 목표 플로우. 오른쪽 그림의 화살표 네 개가 이 문서의 질문 전부입니다 — 신원 확인(Q2), 데이터 전송(Q1·Q3), 접수번호 회신(Q3), 그리고 이것을 어떤 순서로 만들 것인가(Q5).

app.js — 신청 버튼의 진입점

```
function applyTo(sch) {
  if (formTplIdFor(sch)) { startFormFill(sch); return; } // 양식이 있으면 작성 화면
  if (essayDefsFor(sch).length) { startDocPrep(sch); return; }
  finalizeApply(sch, null); // 여기서 끝 — 전송은 없음
  closeSheet();
}
```

data.js — 접수 채널 판정 (마지막 줄이 문제의 지점)

```
function submitChannelLabel(sch) {
  if (sch.applyEmail) return '✉ 이메일 접수 — 앱에서 메일 자동 완성';
  if (sch.program || /한국장학재단/...) return '🏠 한국장학재단 — 본인 인증 후 직접 신청';
  if (sch.formId) return '📄 양식 제출형 — 앱에서 원본 양식 작성';
  if (hasFormAttachment(sch)) return '📎 원본 양식 다운로드형 — 첨부 양식 작성';
  return '💻 온라인·포털 입력형 — 복사해서 붙여넣기'; // ← 나머지 전부가 여기서
}
```

이 다섯 줄이 실측 조사에서 드러난 결함입니다

마지막 줄은 “모르는 것”과 “포털형인 것”을 같은 문구로 처리하고 있었습니다. 121건 조사 결과 실제 포털형은 18건인데, 앱은 접수 방법을 확인하지 못한 공고에도 “포털에서 복사해 붙여넣으세요”라고 단정했습니다. 확인하지 않은 것을 확인한 것처럼 말하는 것은 이 서비스의 원칙 위반이라 **2026-09-21에 고쳤습니다** — 근거가 없으면 “접수 방법은 원문 공고에서 확인”이라고 적습니다. 이 사례를 남겨 두는 이유는, 이런 자리가 여기 하나였을 리 없기 때문입니다. Q4에서 같은 유형을 더 짚어 주시길 요청드립니다.

2.10 이미 만들어 둔, 아직 켜지 않은 자산

연동의 첫 단추가 될 수 있는 코드가 이미 있습니다. **키가 없어 한 번도 실제로 돌려 본 적이 없습니다.**

server/mail-worker.js - 접수 메일 발송 (Cloudflare Worker · 미배포)

```
const cors = { 'Access-Control-Allow-Origin': 'https://seonju5543-web.github.io', ... };

/* 남용 방지 ① 앱에서 온 요청만 (Origin 검사) */
if (origin !== 'https://seonju5543-web.github.io') return new Response('forbidden', { status: 403 });

/* 남용 방지 ② 수신자는 접수처 도메인만 (스팸 릴레이 차단) */
const ALLOWED_TO = /^[^\s@]+@[a-z0-9-]+\.(ac\.kr|or\.kr|go\.kr|re\.kr)$/i;
if (!ALLOWED_TO.test(toClean)) return new Response('recipient not allowed', { status: 403 });

/* 남용 방지 ③ 크기 제한 (본문 100KB · 첨부 5개 각 5MB) */
...
await fetch('https://api.resend.com/emails', {
  headers: { Authorization: `Bearer ${env.RESEND_KEY}` },
  body: JSON.stringify({ from: '한대장 접수대행 <apply@handaejang.app>', to: [toClean],
    reply_to: replyTo, subject, text, attachments })
});
```

발신 도메인 인증·바운스 처리·전송 증빙 보관이 아직 없습니다. **Q5에서 이 세 가지를 여쭙습니다.** 참고로 이 워커에는 저장소(KV)가 없어, 지금 구조로는 “보냈다”는 기록이 어디에도 남지 않습니다.

03 문제 — 실측 121건

2026-09-02, 경희대·한국외대 장학 게시판을 직접 열어 공고 121건(경희 58 · 외대 63)을 받고, 각 공고에서 **신청·접수 방법을 말하는 줄만 원문 그대로** 뽑아 채널을 갈랐습니다. 추론하지 않았고, 근거가 없으면 “미확인”으로 남겼습니다. 경희대는 본문이 “첨부파일 확인 바랍니다”로 끝나 첨부 원본 41개를 받아 글자를 뽑아야 분류가 됐습니다.

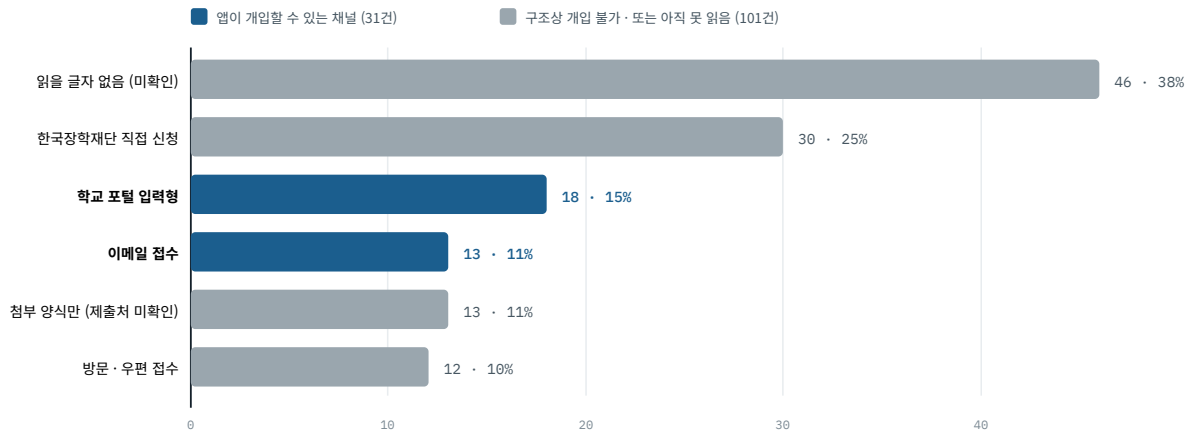


그림 6 신청 채널 분포. 한 공고가 두 채널을 함께 안내하는 경우가 있어 합이 121을 넘습니다. 회색은 우리가 손댈 수 없거나(재단 접수·방문) 아직 판독하지 못한 공고입니다.

3.1 포털형 18건의 실제 경로

포털은 학교마다, 심지어 **같은 학교 안에서도** 다릅니다. 한국외대는 시스템이 둘이고, 그중 하나만 쓰는 장학금이 1건 있습니다. 한 쪽만 보면 “포털 신청이 없다”고 잘못 판단하기 쉬운 구조입니다.

표 5 - 포털형 18건의 경로 (공고 원문에서 발췌)

시스템	건수	원문에 적힌 경로	비고
HUFS Ability (한국외대)	11	로그인 (학번/비번) → 교과/비교과 → 장학신청 목록 → 해당 장학금 선택	11건 전부 같은 경로. 8건이 교외 재단
종합정보시스템 (한국외대)	1	로그인 → 등록/장학정보 → 면학장학금 신청	이 1건만 다른 시스템
인포21 (경희대)	6	신청방법 : 인포21 / 인포21 장학신청 메뉴로 신청	경로를 길게 적지 않음. 2건이 교외 재단
합계	18	이 중 10건이 교외 재단 장학금 — 학교 시스템이 외부 재단의 접수 창구 노릇까지 하고 있습니다	

연동의 값어치를 결정하는 사실

학교 시스템과 다리를 하나 놓으면 **교내에 그치지 않고 교외 재단 장학금까지 함께 열립니다**. 총동문화·남가주동문·가송재단·유흥수·동문홍보대사·라성정형기·미래인재육성(외대), 우덕재단·LEE&MOON·쌍용곰두리(경희) — 재단이 학교에 접수를 위탁한 형태입니다. 즉 “학교 1곳 연동 = 장학금 1종”이 아니라 “학교 1곳 = 그 학교를 창구로 쓰는 모든 재단”입니다.

3.2 구현 후보 다섯 — 우리가 정리한 지형



그림 7 구현 후보 포지셔닝. 가로축은 학교의 협조가 얼마나 필요한가, 세로축은 학생이 손으로 하던 일을 앱이 얼마나 대신하는가입니다. 왼쪽 위가 “허락 없이 지금 만들 수 있고 효과도 있는” 자리이며, 이메일 자동 발송이 여기에 가장 가깝습니다.

표 6 — 다섯 후보에 대한 우리 추정 · 빈칸이 여쭙는 부분

후보	우리 쪽 작업	학교 쪽 필요	우리 추정 공수	모르는 것 (← 고문 요청)
반클릭 딥링크	공고별 정확한 경로·딥 링크 저장, 값 복사 UI	없음	2~3주	로그인 후 특정 화면으로 보내는 딥링크가 학교 포털에서 실제로 유지되는가 / 자동 채움을 어디까지 구현 가능한가
이메일 자동 발송	도메인·발신 인증, 바운스 처리, 전송 증빙	접수 주소 확인 · 수신 인정	1~2주	KV 없는 서버리스에서 전송 증빙을 어떻게 남기나 / 첨부 규격(HWP·PDF 병합) / 도달률
파일 배치	정기 산출·전송, 스키마 확정	수신함 + 취입 배치	3~4주	상시 서버 없이 SFTP를 어떻게 다루나 / 재전송·정정의 표현 방법
연계 API + SSO	토큰 보관, 재시도, 상태 동기화	API 개발 + 계정 발급	6~10주	클라이언트 시크릿을 어디에 두나 / IP 고정을 서버리스에서 어떻게 / 표준 설계
학생 계정 대행	브라우저 자동화 유지 보수	없음(목인 전제)	지속 불가로 판단	이 판단이 맞는가 / 학생 기기 안에서만 도는 형태라면 달라지는가

04 질문 다섯 — 학교 연동

각 질문은 **답에 따라 우리가 만들 물건이 실제로 달라지는 것만** 골랐습니다. 질문마다 ① 여쭙는 문장 ② 우리 상황 ③ 우리 가설 ④ 받고 싶은 답의 형태를 붙였습니다. ③을 적은 이유는, 저희 판단이 틀렸다면 그것을 먼저 알려 주시는 것이 가장 큰 도움이기 때문입니다.

질문 01 · 연동 아키텍처

상시 서버가 없는 구조에서, 학교 시스템에 신청을 넣는 현실적인 방법

저희 구조는 정적 파일 + 이벤트 기반 함수가 전부입니다(그림 2). 상시 구동 서버가 없고, 고정 IP도, 항상 열려 있는 수신 포트도 없습니다. 이 제약 아래에서 학교 장학 시스템에 신청 데이터를 넣는 다섯 후보(그림 7·표 6) 중 실제로 감당 가능한 것은 무엇이고, 순서를 어떻게 잡아야 하나?

특히 여쭙고 싶은 것은 “연계 API를 쓰게 됐을 때 우리 쪽이 무엇을 새로 갖춰야 하는가”입니다. 학교가 흔히 요구하는 접속 IP 고정·상호 TLS·클라이언트 시크릿 보관을 Cloudflare Workers 같은 서버리스에서 다루는 표준적인 방법이 있습니까, 아니면 이 시점에 상시 서버를 한 대 두는 것이 정석입니까?

우리 상황

- 앱은 브라우저에서만 돌립니다. 서버 쪽 코드는 Cloudflare Worker 4개(각 100줄 안팎)가 전부이고, 그중 가동 중인 것은 푸시 발송 1개입니다.
- 학교 시스템은 공개 API도 문서도 없습니다. 경희대 인포21, 한국외대 HUFS Ability는 각각 학사 포털의 일부이고 납품사가 다를 가능성이 큼니다.
- 데이터 흐름은 지금까지 한 방향이었습니다(로봇이 읽어 오기만 함). 학교로 쓰는 방향은 이번이 처음입니다.

우리 가설 — ① 이메일 자동 발송이 학교 개발 0으로 시작할 수 있는 유일한 실제 접수 경로다. ② 연계 API는 학교 예산·발주가 필요해 마지막이다. ③ 파일 배치(정기 CSV 전송)가 중간 지점인데, 상시 서버 없이 SFTP를 다루는 방법을 저희가 모릅니다. — 이 세 가지 중 틀린 것이 있으면 그것부터 알려 주십시오.

받고 싶은 답의 형태

- 다섯 후보에 대한 우선순위와 그 이유. 저희 표 6의 공수 추정이 현실과 얼마나 다른지도 함께.
- 서버리스에서 “학교가 요구하는 접속 조건”을 만족시키는 구체적 구성(또는 그것이 불가능하다는 판단과 대안).
- 학교 학사시스템과 붙어 본 경험이 있으시다면, 그때 실제로 쓴 방식과 가장 오래 걸린 지점.
- 저희가 후보 목록에서 빠뜨린 방식이 있다면 그것.

질문 02 · 인증과 세션

학교 계정 로그인 벽을 넘는 표준 설계

포털형 18건은 전부 **학번·비밀번호 로그인** 뒤에 있습니다. 학교가 SSO를 열어 준다는 전제에서, **정적 PWA + 서버리스 함수 + Supabase**라는 저희 구조에 대학 SSO(OIDC 또는 SAML)를 붙이는 표준 설계는 무엇입니까?

구체적으로 막히는 지점 셋입니다. ① 토큰을 어디에 두어야 합니까 — 지금 로그인 토큰은 `localStorage`에 있는데 이대로 괜찮습니까. ② 서비스워커는 로그인을 **모르도록** 일부러 설계했는데(백그라운드 알림이 서버에 붙는 경로를 만들지 않기 위해), SSO를 붙이면 이 경계가 유지됩니까. ③ Supabase 계정과 학교 IdP 신원, 두 개의 신원을 어떻게 잇습니까 — 어느 쪽을 주(主) 신원으로 삼아야 합니까.

우리 상황

- CSP가 `script-src 'self'`라 외부 SDK를 스크립트로 불러올 수 없습니다. Supabase도 SDK 없이 `fetch` 4개로만 씁니다. SSO 라이브러리가 필요하면 이 정책을 손대야 합니다.
- 앱은 설치형 PWA입니다. 리다이렉트 기반 로그인이 설치된 앱에서 어떻게 동작하는지가 저희에겐 미지수입니다.
- 대안으로 **학생 기기 안에서만 도는 자동 입력**(브라우저 확장·인앱 웹뷰)을 생각해 봤지만, 유지비와 실현성을 판단할 근거가 없습니다.

우리 가설 — 학생의 학교 아이디·비밀번호를 우리가 받아 대신 로그인하는 방식은 배제해야 한다(그림 7의 ✕). 자격증명을 보관하는 순간 사고의 크기가 달라지고, 학교가 알면 즉시 차단할 것으로 보입니다. 다만 **학생 기기 밖으로 자격증명이 나가지 않는 형태**라면 판단이 달라지는지는 저희가 모릅니다.

받고 싶은 답의 형태

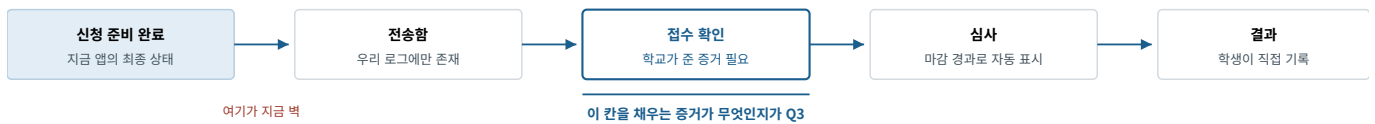
- SSO를 붙일 때의 **구성도 한 장 수준**의 설계(어디서 리다이렉트를 받고, 토큰을 어디에 두고, 누가 검증하는지).
- 학교에 SSO 연동을 요청할 때 **개발자가 준비해 줘야 하는 항목 목록**(리다이렉트 주소, 스코프, 인증서, 테스트 계정 등).
- “이 방식만은 하지 마라”에 해당하는 것 하나와 그 이유.
- 학교가 SSO는 못 열고 **학번 대조만** 해 준다면, 그것으로 접수까지 갈 수 있는 설계가 있는지.

질문 03 · 데이터 계약과 먹등성

무엇을 보내고, 무엇을 받아야 ‘접수됐다’고 말할 수 있는가

저희에게는 이미 48종 · 602필드의 신청서 스키마와 33값의 학생 프로필이 있습니다(2.6·2.7). 이것을 학교가 받아먹을 수 있는 형태로 옮길 때, 스키마를 어떻게 설계해야 합니까? 그리고 전송의 신뢰성 — 재전송·중복 제출·정정·취소·부분 실패 — 은 어떤 패턴으로 다뤄야 합니까?

특히 접수 확인을 어떻게 받을 것인가가 핵심입니다. 저희는 “실제 접수가 일어나지 않은 것을 신청 완료라고 쓰지 않는다”를 원칙으로 지켜 왔습니다. 접수번호 즉시 반환 · 상태 조회 · 콜백 통지 · 야간 배치 결과 파일 중 상시 서버가 없는 저희가 실제로 받을 수 있는 형태는 무엇입니까? 콜백을 받으려면 결국 상시 수신 창구가 필요한 것 아닙니까?



지금 규칙 — ‘심사’ 단계만 객관적 사실(제출 기록 + 마감 경과)로 자동 표시하고, 제출·결과는 학생이 직접 누릅니다.

학교 전산을 볼 수 없기 때문입니다. 연동이 되면 ‘전송함 → 접수 확인’ 두 칸이 처음으로 자동이 됩니다.

그림 8 신청 상태 머신. 파란 칸을 무엇으로 채울 수 있느냐가 “신청 완료”라고 표기해도 되는지를 가릅니다.

우리 가설 — 먹등성 키를 붙여 재전송해도 중복 접수가 되지 않게 하고, 학교 응답이 늦으면 폴링으로 상태를 확인하는 구조가 맞다고 봅니다. 다만 **폴링을 누가 도느냐**가 문제입니다 — 앱은 학생이 열어야만 돌고, 저희에게 상시 프로세스가 없습니다. GitHub Actions 스케줄(하루 몇 회)로 대신하는 것이 정상적인 해법인지 판단이 서지 않습니다.

받고 싶은 답의 형태

- 학교에 제안할 **최소 데이터 계약**의 뼈대 — 필드 수준까지는 아니어도, 무엇을 반드시 포함해야 하는지.
- 중복·정정·취소·부분 실패를 다루는 **패턴 이름과 최소 구현**(먹등성 키, 아웃박스, 상태 머신 등).
- 상시 서버 없이 접수 확인을 받는 현실적인 방법. 없다면 “여기서부터는 서버가 필요하다”는 선을 그어 주시면 됩니다.
- 학생이 “냈는데 누락됐다”고 다룰 때 **기술적으로 남겨 두어야 할 기록**이 무엇인지.

질문 04 · 구조 리뷰

이대로 학교와 붙이면 터질 곳은 어디입니까

2장에 앱의 구조·코드·데이터를 전부 적었고 저장소도 공개입니다. 연동을 시작하기 전에 반드시 고쳐야 할 것을 짚어 주십시오. 저희가 스스로 의심하는 자리 여섯을 아래에 적었습니다 — 이 목록이 맞는지, 순서가 맞는지, 빠진 것이 있는지 봐 주시면 됩니다.

우리가 의심하는 자리 — 위험하다고 보는 순서대로

#	자리	왜 걱정되나
1	모든 판단이 클라이언트에	자격 판정·마감·금액 계산이 전부 브라우저에서 돛니다. 서버 검증이 없어, 조작된 신청이 학교로 넘어가는 것을 막을 지점이 없습니다.
2	메일 워커의 방어가 Origin 검사뿐	브라우저가 아닌 곳에서 오는 요청은 Origin을 위조할 수 있습니다. 수신 도메인 화이트리스트가 있지만, 접수처로 스팸을 보내는 통로가 되지 않는지 확인이 필요합니다.
3	전송 기록이 남지 않음	워커에 저장소가 없어 “보냈다”는 기록이 어디에도 없습니다. 분쟁이 나면 증명할 수단이 없습니다.
4	로봇이 데이터 원본을 매일 덮어씀	사람이 등록한 데이터와 로봇이 쓰는 파일이 같습니다. 병합 규칙으로 막고 있지만, 학교와 주고받는 데이터가 여기 섞이면 사고의 성격이 달라집니다.
5	정적 파일 통째 다운로드	공고 목록 34 KB를 앱이 통째로 받아 자기 학교 것만 고릅니다. 학교가 늘면 한계가 옵니다 (실측 기준 50~100개교).
6	서비스워커 캐시 버전 수동 관리	현재 v145. 올리는 것을 잊으면 옛 화면이 남습니다(실제로 두 번 겪었습니다). 네트워크 우선이라 결국 반영되지만 3.5초를 넘기면 캐시로 떨어집니다.

우리 가설 — 1번(서버 검증 없음)이 학교와 붙는 순간 가장 크게 문제가 될 것으로 봅니다. 지금까지는 “틀려도 학생 화면만 틀리는” 구조였는데, 연동이 되면 **틀린 데이터가 학교 시스템에 들어갑니다**. 다만 서버 검증을 넣으면 “고정비 0원 · 상시 서버 없음”이라는 구조가 깨지므로, 어느 정도까지가 적정선인지 판단이 필요합니다.

받고 싶은 답의 형태

- 위 여섯 개의 **재정렬**과, 저희가 못 본 항목 추가.
- 각 항목에 대해 “연동 전에 반드시” / “연동과 함께” / “나중에” 세 등급으로만 나눠 주셔도 충분합니다.
- 1번에 대한 **최소 처방** — 상시 서버를 두지 않고 검증을 넣는 방법이 있는지.
- 코드를 직접 보실 수 있으면, 저희가 **안전하다고 믿고 있는데 실은 아닌** 자리를 하나만이라도 짚어 주십시오.

질문 05 · 90일 실행 계획

협약 전에, 지금 무엇을 만들어야 합니까

학교와의 협약은 시간이 걸립니다. 그동안 **학교 개발 없이** 할 수 있는 것이 둘 있고, 이 요청서를 쓴 뒤 **둘 다** 일부를 먼저 만들었습니다(2026-09-23).

① **이메일 접수** — 공고 원문에서 접수 주소를 읽어 앱에 담았고, 누르면 학생 폰의 메일 앱이 제목·본문이 채워진 채 열립니다(mailto: — 서버도 열쇠도 필요 없었습니다). 남은 것은 **앱이 대신 보내는 것**이고 거기서부터 발송 서버가 필요합니다. ② **포털** — 공고가 말한 시스템까지 읽어(한 학교가 시스템을 둘 쓰는 것을 여기서 알았습니다) 그 시스템으로 보내고 원문의 경로 문장을 그대로 보여 줍니다. 남은 것은 **값을 미리 갖춰 두는 것**입니다.

그래서 **여쭙고 싶은 것은 “다음 한 칸”입니다** — 이 둘을 어디까지 밀어야 하고, 어느 지점부터는 학교 협약 없이는 의미가 없어집니까?

①의 기술 요건을 특히 여쭙습니다 — **발신 도메인 인증(SPF·DKIM·DMARC)**, **바운스 처리**, **전송 증빙 보관**. 저희 워크에는 저장소가 없어 증빙을 남길 자리가 없고, 학교 메일 서버가 외부 도메인 첨부 메일을 어떻게 취급하는지도 모릅니다. ②는 **자동 채움**을 어디까지 구현할 수 있는지가 관건입니다 — **클립보드 API**, **딥링크 파라미터**, **PWA 공유 대상** 중 실제로 쓸 만한 것이 있습니까?

우리 상황 — 이미 있는 재료

- 발송 코드(server/mail-worker.js)는 완성돼 있습니다. Origin 검사 · 수신 도메인 화이트리스트 · 크기 제한까지 들어 있고, **키가 없어 한 번도 돌려 본 적이 없습니다**. ⚠️ 다만 이것은 **완전 자동 발송**용입니다 — 학생이 직접 보내는 길(mailto:)은 이미 열려 있고 서버를 타지 않습니다.
- 앱이 만드는 첨부는 .doc(Word HTML) 형식입니다. 학교가 HWP를 기대할 때 무엇으로 보내야 하는지 판단이 필요합니다.
- 포털형의 **정확한 경로 문장은 이미 확보**돼 있고(표 5), 이제 **앱 데이터에도 들어가 학생 화면에 원문 그대로 보입니다**. 딥링크로 만들 재료는 있습니다.

우리 가설 — ①을 먼저 하는 것이 맞다고 봅니다. 학교에 보여 줄 것이 생기고(“이미 돌아갑니다”), 코드가 이미 있어 가장 빠르며, 무엇보다 **“신청 준비 완료”를 “접수 완료”로 바꿀 수 있는 유일한 자리**이기 때문입니다. ②는 효과가 확실하지만 학생의 손을 완전히 덜어 주지는 못합니다.

받고 싶은 답의 형태

- **90일을 3구간으로** 나눈 계획 — 무엇을 언제까지, 무엇을 뒤로 미룰지.
- 메일 발송을 “실제로 접수로 인정받는” 수준까지 올리는 **체크리스트**(도메인·인증·첨부·증빙·실패 처리).
- 반클릭 자동 채움의 **실현 가능한 최대치**와, 넘어서면 안 되는 선.
- 학교에 보여 줄 **데모의 형태** — 무엇을 시연하면 개발자가 아닌 사람에게 설득력이 있는지.

05 질문 일곱 — 지금의 구조

앞의 다섯이 **학교와 어떻게 이룰 것인가**라면, 이 일곱은 **지금 만들어 둔 것이 그 무게를 견디는가**입니다. 연동을 시작하면 이 구조 위에 학교 데이터가 얹히므로, 먼저 여쭙는 편이 낫다고 판단했습니다. 형식은 앞과 같습니다 — ① 여쭙는 문장 ② 우리 상황 ③ 우리 가설 ④ 받고 싶은 답의 형태.

질문 06 · 회원 데이터 모델

프로필과 신청내역을 jsonb 한 칸에 통째로 넣어도 됩니까

회원 데이터를 표 하나(**profiles**)에 넣고, 학생 프로필 33값과 신청내역 전체를 각각 jsonb 한 칸에 통째로 저장하고 있습니다. 이대로 가도 됩니까, 아니면 지금이라도 항목별로 칸을 쪼개야 합니까? **쪼개야 한다면 무엇을 기준으로 어디까지 쪼개는 것인지** 알고 싶습니다.

쪼개지 않은 이유는 앱의 프로필 구조가 이미 두 번 바뀌었기 때문입니다(분교·캠퍼스 구분 도입, 적합도 필드 추가 — 앱 안에 이전 코드가 남아 있습니다). 칸으로 쪼개면 앱을 고칠 때마다 DB 설계도 함께 고쳐야 하고, 안 고치면 값이 조용히 사라집니다. 다만 이 선택이 나중에 어떤 대가로 돌아오는지를 저희가 모릅니다.

우리 상황

- 표는 하나입니다 — user_id(기본키) · profile jsonb · applications jsonb · sensitive_ok boolean · updated_at. 그 외 표는 없습니다.
- 방어선은 **행 단위 접근 규칙(RLS) 하나뿐**입니다 — “auth.uid() = user_id일 때만”. 즉 서버는 “누구의 행인가”만 보고, 칸 안의 내용은 전혀 검증하지 않습니다.
- 동기화는 **행 통째로 최신이 이깁니다**(updated_at 비교 · last-write-wins). 저장할 때마다 보내지 않고 2초 묶어 한 번 upsert 합니다.
- 기기가 원본이고 서버는 사본입니다. 주민등록번호·계좌번호는 아예 올라가지 않고, 특별자격은 동의했을 때만 올라갑니다(그림 4).

우리 가설 — 지금 규모에서는 jsonb 통짜가 맞고, **자주 찾는 값 몇 개(학교·캠퍼스 정도)만 따로 빼 두면** 충분하다고 봅니다. 그러나 두 가지가 걸립니다. ① **두 기기에서 각각 다른 곳을 고치면 한쪽 변경이 통째로 사라집니다** — 행 전체를 덮어쓰기 때문입니다. ② 서버가 내용을 안 보므로, 학생이 임의의 값을 넣어도 막을 지점이 없습니다. 연동이 되면 이 값이 학교로 넘어갑니다.

받고 싶은 답의 형태

- “**여기까지는 jsonb, 여기부터는 칸**”의 선. 판단 기준까지 주시면 저희가 이후에 스스로 적용하겠습니다.
- 칸 단위 병합이 **반드시 필요해지는 시점**의 신호 — 그 전에는 안 해도 되는지.
- 내용 검증을 어디에 두어야 합니까 — DB 제약(CHECK)인지, 서버 함수인지, 아니면 클라이언트로 충분한지.
- 나중에 쪼개야 할 때 **이미 쌓인 데이터를 옮기는 방법**(무중단이 가능한지).

질문 07 · 확장성

사용자가 늘면 가장 먼저 터지는 곳은 어디입니까

지금은 사용자가 사실상 없습니다. 학생이 100명, 1,000명, 10,000명이 됐을 때 어느 층이 가장 먼저 무너집니까? 저희는 짚이는 곳이 몇 군데 있지만, 그것이 정말 첫 번째인지 판단할 근거가 없습니다.

함께 여쭙고 싶은 것은 “터지기 전에 어떻게 알아채는가”입니다. 지금은 무엇이 얼마나 쓰이는지 재는 장치가 없어서, 문제가 생기면 학생이 알려 주기 전까지 모릅니다. 상시 서버가 없는 구조에서 계측을 어디에 어떻게 붙여야 합니까?

우리 상황 — 저희가 아는 한계값

- **공고 목록을 통째로 내려받습니다.** 앱은 notices.json 34 KB를 전부 받아 자기 학교 것만 고릅니다. 학교별로 파일이 나뉘어 있지 않아, 실측 기준 **50~100개교**가 한계입니다(현재 등록 2개교).
- **GitHub Actions** — 공개 저장소인 동안은 무제한이라 제약이 아닙니다. 비공개로 전환하면 그 순간 월 2,000분이 상한이 되고, 현재 사용량은 약 700분입니다.
- **Supabase·Cloudflare 모두 무료 등급**입니다. 어느 지표가 먼저 한도에 닿는지 저희는 모릅니다.
- **푸시 발송**은 2분마다 25대씩 보냅니다 — 수백 명까지는 무해하다고 계산했지만 실제로 겪어 본 적은 없습니다.
- **공고 검수는 사람이 합니다.** 애매한 공고는 자동 등록하지 않고 개발자가 직접 판단합니다(그림 3).

우리 가설 — 기술보다 사람이 먼저 막힌다고 봅니다. 서버는 사용자가 늘어도 거의 그대로인데(판단이 전부 학생 기기 안에서 일어나므로), 공고 검수는 학교가 늘수록 선형으로 늘고 한 명이 합니다. 기술 쪽에서 첫 번째는 **공고 목록 통째 다운로드**라고 보고, 학교가 40곳쯤 될 때 학교별 파일로 나눌 계획입니다. 이 순서가 맞습니까?

받고 싶은 답의 형태

- 무너지는 **순서**와, 그렇게 보시는 이유. 저희 가설에서 틀린 것이 있으면 그것부터.
- 100명 · 1,000명 · 10,000명 **각 시점에 무엇이 달라지는지** — 특히 “이때는 구조를 바꿔야 한다”는 지점.
- 상시 서버 없이 **계측을 붙이는 최소 구성**(무엇을 재야 하고, 어디에 남겨야 하는지).
- 저희가 **과하게 걱정하고 있는 것**이 있다면 그것도 알려 주십시오 — 안 해도 될 일에 시간을 쓰고 싶지 않습니다.

질문 08 · SDK 없이 직접 호출

보안 정책을 지키려고 SDK를 안 쓰는데, 그 대가가 무엇입니까

로그인·동기화에 Supabase를 쓰지만 공식 SDK를 쓰지 않고 주소 4개를 fetch로 직접 부릅니다. 앱의 보안 정책이 `script-src 'self'`라 외부 스크립트를 실행할 수 없기 때문입니다 — 그 차단은 “수집한 공고 데이터가 오염돼도 코드 실행으로 이어지지 않는다”는 이 앱의 보안 근거 그 자체라, 풀고 싶지 않았 습니다. 이 선택의 위험은 무엇입니까?

SDK가 대신 해 주던 일을 저희가 직접 짰습니다 — 토큰 저장, 만료 1분 전 갱신, 401이면 한 번 갱신 후 재시도. 특히 의 심스러운 자리가 하나 있습니다: Supabase는 갱신 토큰을 한 번 쓰면 새것으로 바꾸는데(회전), 탭이 여러 개이거나 여 러 요청이 동시에 만료를 맞으면 각자 갱신을 시도해 서로의 토큰을 무효로 만들고 로그인에 통째로 풀리지 않습니까? 이것이 실제 결함인지, 그렇다면 fetch만으로 어떻게 막는지 알고 싶습니다.

우리 상황

- 부르는 주소는 넷뿐입니다 — 가입/로그인/토큰갱신(/auth/v1/...), 내 정보, 로그아웃, 프로필 읽기·쓰기 (/rest/v1/profiles). 외부 의존성은 0개입니다.
- 토큰(접속·갱신 둘 다)은 localStorage에 둡니다. 비밀번호는 저장하지 않고, 이메일 한 줄만 ‘아이디 저장’으로 남깁니다.
- 서비스워커는 로그인을 모릅니다 — 백그라운드 알림이 서버에 붙는 경로를 만들지 않으려고 일부러 그렇게 했습니다.
- 공개 열쇠(anon key)는 브라우저에 그대로 나갑니다. 숨길 수 없는 값이라, 실제 방어는 RLS 하나에 걸려 있습니다(질문 06).

우리 가설 — 보안 정책을 지키는 값이 SDK의 편의보다 크다고 보아 이 선택을 유지하려 합니다. 다만 **동시 갱신 문제는 진짜 결함일 가능성 이 높다**고 의심하고 있고, 갱신 토큰을 localStorage에 두는 것도 마음에 걸립니다. 그리고 라이브러리가 조용히 고쳐 주던 것(주소 변경 ·보안 패치)을 저희는 받지 못한다는 점이 장기적으로 어떻게 돌아올지 모르겠습니다.

받고 싶은 답의 형태

- 이 방식의 **진짜 위험을 순서대로** — 저희가 걱정하는 것과 실제로 위험한 것이 다르다면 그것부터.
- 동시 갱신을 **fetch만으로 막는 표준 방법**(있다면 이름만 알려 주셔도 저희가 찾아 구현하겠습니다).
- 토큰을 어디에 두어야 합니까 — 지금 위치가 괜찮은지, 아니면 옮겨야 하는지.
- 보안 정책을 **깨지 않으면서** SDK를 쓰는 길이 있는지(예: 직접 묶어서 자체 파일로 배포).

질문 09 · 수집 범위

수집망을 넓히는 판단은 무엇을 보고 내려야 합니까

김성민 교수님께서 “외부 공고를 전부 수집하는 것이 좋다”고 조언해 주셨습니다. 저희는 반대로 좁혔습니다 — 2026-09-21에 수집 대상을 **경희대·한국외대 2곳으로 확정**하고 나머지 42개교를 내렸습니다. 온보딩에서는 213개교를 고를 수 있으므로 **다른 학교 학생에게는 앱이 비어 보입니다**. 이것은 사고가 아니라 **알고 내린 결정**입니다: 공고가 늘수록 못 읽은 자격 요건과 잘못 만든 신청서 양식이 학생 화면에 그대로 나가는데, 그 오류를 사람이 다 볼 수 없기 때문입니다.

여쭙고 싶은 것은 “**넓혀도 되는가**”가 아니라 “**언제 · 무엇을 보고 넓히는가**”입니다. 지금의 좁힘이 옳은 판단인지, 그리고 넓혀도 된다고 말해 주는 신호가 무엇인지 알고 싶습니다.

비용은 제약이 아닙니다 — 공개 저장소인 동안 로봇 실행은 무제한 무료입니다(비공개로 돌릴 때만 월 2,000분이 상한이 되고, 현재 700분입니다). 걸리는 것은 **정확도 하나**이고, 그래서 넓히기를 **미룬 것**입니다.

우리 상황

- 지금도 자격 요건과 양식 스키마화는 **공고 원문의 구조에 크게 좌우됩니다**. 절 머리글이 없거나 첨부 안에만 내용이 있으면 판독률이 떨어집니다.
- 이미 **2층 구조를 쓰고 있습니다** — 한국장학재단 재단 101곳은 목록과 링크만 보여 주고, 자격 진단·양식 작성을 붙이지 않습니다. 재단이 적어 둔 칸을 그대로 옮길 뿐입니다.
- 잘못된 데이터는 **감사 관문**이 막습니다(그림 3). 다만 이 관문은 “규칙에 어긋나는가”를 볼 뿐, “**원문을 제대로 읽었는가**”는 보지 못합니다.
- 공고가 늘면 앱이 내려받는 파일도 함께 커집니다(질문 07의 첫 번째 한계값).

우리 가설 — 넓힐 때가 오면 새로 들어오는 공고는 **정식 등록(매칭·자격 진단·양식 작성)에 넣지 않고 ‘목록·링크만 보여 주는 층’으로 따로 둔다**는 것이 저희 안입니다. 자격 진단과 양식 작성은 사람이 원문을 검수한 공고에만 붙입니다. 이렇게 하면 수집을 몇 배로 늘려도 **틀린 판정이 학생에게 나가지 않습니다** — 대신 대부분의 공고에서 앱이 해 주는 일이 “있다고 알려 주는 것”뿐이 됩니다. 이 맞바꿈이 옳은지, 그리고 **그 구조가 준비되기 전에는 넓히지 않는 것이 맞는지** 판단이 서지 않습니다.

받고 싶은 답의 형태

- **넓혀도 된다고 말해 주는 신호** — 무엇이 갖춰지면(또는 어떤 숫자가 어디에 닿으면) 2곳을 늘려도 된다고 볼 수 있습니까. 지금의 좁힘을 언제까지 끌고 가야 합니까.
- 이 **2층 구조가 맞는지**, 아니면 다른 방식이 있는지.
- 대량 수집에서 **품질을 자동으로 재는 방법** — 사람이 다 볼 수 없는데 무엇이 틀렸는지 알아야 합니다(수집·판독의 구체적인 어려움은 질문 10·11에 나눠 적었습니다).
- 크롤링을 그만두고 **제휴·제공받기로 넘어가야 하는 지점**이 있다면 그 신호.
- 남의 사이트를 대량으로 읽을 때 **넘으면 안 되는 선**(접속 빈도, robots.txt, 원문 재게시 범위 등) — 기술 관행 수준에서.

질문 10 · 수집 차단

사이트마다 막는 방식이 다른데, 유형별로 어떻게 접근해야 하나까

광고 수집은 Node.js 로봇이 하는데, 사이트마다 외부 접근을 막는 방식이 전부 다릅니다. 어떤 곳은 그냥 HTML을 받으면 되고, 어떤 곳은 진짜 브라우저를 띄워야 하며, 어떤 곳은 자주 두드리면 막습니다. 사이트 유형을 어떻게 나누고, 각 유형에 어떤 도구로 접근하는 것이 표준입니까?

특히 어려운 것은 “막힌 것”과 “느린 것”과 “원래 없는 것”을 구분하는 일입니다. 실제로 한 학교의 광고 12건을 로봇이 “다른 글이 열렸다”며 전부 버린 적이 있는데, 직접 열어 보니 주소는 처음부터 전부 맞았고 학교가 접근을 제한하면서 404가 아니라 본문이 안 그려진 겹데기 화면을 돌려주고 있었습니다. 로봇이 그것을 “다른 글”로 읽은 것입니다.

우리 상황 — 지금까지 만난 유형

- 보통 게시판 — HTML을 받아 읽으면 됩니다. 대부분이 여기 속합니다.
- 클릭해야만 열리는 게시판 — 목록의 행이 진짜 링크가 아니라 스크립트이고, 상세 주소가 전부 같습니다. 지금은 진짜 Chromium으로 행을 실제 눌러 상세 화면에서 바로 채집합니다(8개교).
- 두드리면 막는 게시판 — 간격을 900ms에서 2500ms로 늘렸고, 불일치가 나면 4초 더 기다려 다시 확인합니다. 팝업이 화면을 덮어 클릭이 실패하는 경우도 있어 팝업을 먼저 닫습니다.
- 주소가 매번 바뀌는 게시판 — 상세 주소에 목록에서의 정렬 순번이 붙어, 글이 한 칸만 밀려도 주소가 달라집니다. 어제 본 글을 매일 새 글로 다시 담아 피드가 중복으로 찻습니다(주소에서 휘발성 값을 떼어내 해결).
- brew·명령줄로는 안 열리는 곳 — 자동 제출되는 로그인 페이지나 프레임 구조라, 세션 없는 브라우저로 열면 멀쩡히 보입니다. 이것을 “학생도 못 본다”고 잘못 판단한 적이 있습니다.
- 브라우저를 띄우는 수집은 느리고 시간 예산이 빠듯합니다 — 전체 22분 안에 끝내야 하고, 못 끝낸 학교는 다음 실행으로 넘깁니다.

우리 가설 — “HTML로 되는 곳은 HTML로, 안 되는 곳만 브라우저로”가 맞다고 보고 두 로봇을 따로 두었습니다. 그런데 어느 쪽인지 판정하는 규칙이 없어 사람이 학교마다 눈으로 보고 정하고 있습니다. 그리고 차단당했을 때 우리가 하는 일이 “기다렸다 다시”뿐인데, 이것이 예의 있는 방법인지 아니면 애초에 접근을 다르게 해야 하는지 모르겠습니다.

받고 싶은 답의 형태

- 유형 분류와 그에 맞는 도구 — 저희가 나눈 다섯이 통용되는 구분인지, 이름이 따로 있는지.
- “이 사이트는 어느 유형인가”를 사람이 아니라 로봇이 판정하게 만드는 방법이 있는지.
- 차단·지연·부재를 구분해 내는 표준적인 방법(응답 코드만으로는 안 되는 것을 겪었습니다).
- 남의 사이트를 매일 읽을 때 지켜야 할 관행 — 간격, 동시 접속 수, 신원을 밝히는 방식.

질문 11 · 항목 추출

공고에서 ‘지원 자격’만 뽑아내는 표준적인 방법이 있습니까

공고 원문에서 지원 자격 부분만 뽑으려고 머리글 기호를 정규식으로 찾고 있습니다. ◎ 신청 자격 · 4. 선발기준 · □ 지원 대상 같은 것들입니다. 그런데 공고마다 표현이 달라 계속 어긋나고, 제출 서류·심사 일정·문의처가 자격 칸에 섞여 들어옵니다. 이런 제각각인 문서에서 특정 항목을 뽑아내는 표준적인 도구나 접근법이 따로 있습니까?

저희가 규칙을 하나씩 덧붙이는 방식으로 고쳐 왔는데, 같은 지적을 세 번 받았고 한 유형은 다섯 번 재발했습니다. 원인은 개별 버그가 아니라 방식이었습니다 — 잡음을 이름 대서 하나씩 버리는 목록이라, 새로운 표현은 사람이 눈으로 볼 때까지 아무도 모릅니다. 규칙을 더 쌓는 것 말고 다른 길이 있는지 알고 싶습니다.

우리 상황

- 공고는 자기 구조를 스스로 적어 둡니다(“◎ 신청 자격” / “◎ 지원 제외 대상”). 그래서 낱말이 아니라 **절 경계**로 가르도록 바꿨고, 그 판정은 파일 하나에 모아 화면·추출기·감사가 함께 씁니다.
- 그래도 새는 곳이 남습니다. 예를 들어 ※로 시작하는 줄을 부연 설명으로 보고 버렸는데, **그중 95개가 실제 자격 조건이었습니**다(“※ 졸업유예자·휴학생은 지원 불가”처럼).
- 원본이 **첨부 HWP 안에만** 있는 경우가 많습니다(한 학교는 본문이 “첨부파일 확인 바랍니다”로 끝납니다). HWP에서 글자를 뽑아 쓰는데, 표가 섞이면 줄 순서가 흐트러집니다.
- AI(Claude API)도 쓰고 있지만 **보조로만** 씁니다 — 이 서비스는 “**추론 금지 · 원문 발췌**”가 원칙이라, 없는 문장을 만들어 내면 안 되기 때문입니다.
- 못 읽은 자격은 **학생 화면에서 조용한 결함이 아니라 “틀린 안심”**이 됩니다 — 자격이 안 되는데 된다고 표시되는 일이 실제로 있었습니다.

우리 가설 — 정규식만으로는 끝이 없다고 봅니다. 그렇다고 AI에게 “자격을 뽑아 줘”라고 말기면 **지어낼 수 있어** 원칙과 충돌합니다. 그래서 저희가 쓰는 방식은 “**쓰게 하지 않고 고르게 하는 것**”입니다 — 원문에 줄 번호를 매겨 주고 **어느 줄이 자격인지 번호만 답하게** 합니다. 그러면 없는 문장이 나올 자리가 없습니다. 이 방식이 표준적인 것인지, 아니면 더 나은 길이 있는지 모르겠습니다.

받고 싶은 답의 형태

- 이런 문서에서 항목을 뽑는 **접근법의 이름과 순서** — 저희가 모르는 도구가 있다면 그것.
- “**고르게 한다**”는 방식이 맞는지, 아니면 다른 방법으로도 지어냄을 막을 수 있는지.
- 추출이 **얼마나 맞는지 자동으로 재는 방법** — 지금은 틀린 것을 사람이 봐야만 압니다.
- 완전히 못 읽는 공고가 남을 텐데, 그것을 **어떻게 다루는 것이 옳은지**(감출지, 원문만 보여 줄지).

질문 12 · 개발 방식

AI와 대화하며 만든 코드의 한계선을 어디로 봐야 합니까

이 문서에 적은 것 전부 — 앱, 로봇 47개, 검증 드라이버 42개, 서버 코드까지 — 를 AI와 대화하며 만들었습니다(이른바 바이트 코딩). 속도는 빠르지만, 저희 힘으로는 품질을 판단할 수 없습니다. 이 방식이 어디까지 버티고, 어디부터 사람이 잡아야 합니까?

이 질문을 드리는 이유는 실제로 겪은 두 가지 때문입니다. ① 같은 유형의 버그가 반복해서 돌아옵니다 — 고쳤다고 확인한 것이 몇 주 뒤 다른 모습으로 다시 나왔고, 한 유형은 다섯 번까지 반복됐습니다. ② 검사 자체가 조용히 죽어 있습니다 — 2026-08-29에 세어 보니 검사 31개 중 자동으로 실행되던 것은 5개였고, 손으로 돌려 보니 7개는 이미 깨진 채였습니다. 그동안 화면은 계속 초록불이었습니다.

우리 상황

- 지금의 대응은 “고칠 때마다 그 자리를 지키는 검사를 함께 만든다”입니다. 그래서 검증 드라이버가 42개, 데이터 감사 관문이 저장 직전에 있습니다. 규칙을 되돌리면 검사가 즉시 실패합니다.
- 판정 규칙은 절대 복사하지 않습니다 — 화면·서비스워커·감사 도구가 같은 파일을 불러 씁니다. 규칙이 두 벌이 되어 “화면에서 숨긴 공고를 알림이 알리는” 모순이 실제로 났기 때문입니다.
- 그럼에도 저희는 코드를 읽어 판단하지 못합니다. 화면에 보이는 것과 검사 결과만 보고 믿습니다.
- 앞으로 개발자를 두거나 외주를 줄 가능성이 있습니다. 지금 코드를 다른 사람이 이어받을 수 있는 상태인지 모릅니다.

우리 가설 — 보안·개인정보 처리·데이터 구조는 사람이 설계하지 않으면 나중에 크게 물어뜯길 영역이라고 봅니다(그래서 질문 06·08을 따로 드렸습니다). 반대로 화면·문구는 이 방식으로 계속 가도 된다고 봅니다. 수집 로봇은 판단이 서지 않는 쪽이라 따로 여쭙습니다(질문 10·11) — 매일 도는데 틀려도 조용하기 때문입니다. 그리고 “검사를 함께 만든다”가 규모가 커져도 통하는 방법이라고 믿고 있는데, 이 믿음 자체가 검증된 적이 없습니다.

받고 싶은 답의 형태

- 코드를 읽지 못하는 사람이 품질을 판단하는 방법 — 무엇을 보면 “이건 위험하다”를 알 수 있습니까. 이것이 저희에게 가장 필요한 답입니다.
- 이 저장소에서 “여기는 사람이 다시 설계해야 한다”는 영역 하나만이라도 짚어 주십시오(코드는 공개돼 있습니다).
- 같은 버그가 반복될 때, 검사를 더 만드는 것 말고 다른 처방이 있습니까.
- 나중에 사람에게 넘길 때 주로 무엇이 문제가 되는지 — 지금부터 대비할 수 있게.

06 부록

A. 답변을 받으면 우리가 정할 것

표 7 - 답변이 직접 바꾸는 결정

질문	답이 정하는 것	정해지지 않으면 못 하는 일
Q1	연동 방식 · 우리 쪽 인프라를 바꿀지 여부	학교에 무엇을 요청할지 정할 수 없음
Q2	로그인 구조 — 학교 계정을 붙일지, 기기 안에서 끝낼지	서버 구조 전체가 대기 상태
Q3	앱의 상태 표기 문구 · 학교에 요청할 응답 형식	“신청 완료”라고 쓸 수 있는 조건을 못 정함
Q4	연동 전 수리 목록과 순서	고칠 것을 모른 채 연동을 시작하게 됨
Q5	이번 달 개발 순서	학교 미팅에 보여 줄 것이 없음
Q6	회원 데이터를 지금 쪼갤지, 그대로 둘지	사용자가 쌓인 뒤에는 옮기는 비용이 커짐
Q7	다음에 손볼 곳의 순서 · 계측을 붙일 자리	무엇이 먼저 터질지 모른 채 학생을 받게 됨
Q8	로그인 구현을 고칠지 · SDK로 돌아갈지	세션이 풀리는 결함을 안은 채 연동을 시작함
Q9	수집을 어디까지 넓힐지 · 2층 구조를 유지할지	공고를 늘릴수록 틀린 판정도 함께 늘어남
Q10	학교를 늘릴 때 어느 로봇으로 붙일지	새 게시판마다 사람이 눈으로 보고 정해야 함
Q11	자격 추출 방식을 바꿀지 · 지금 방식을 밀고 갈지	같은 유형의 오류가 계속 돌아옴
Q12	사람이 다시 설계할 영역 · 인수인계 준비	품질을 스스로 판단할 방법이 없는 상태가 계속됨

B. 이 문서에 넣지 않은 질문

아래는 저희가 답이 필요하다고 판단했지만 **개발 고문의 영역이 아니라고 보아 뺐** 것입니다. 혹시 겹치는 경험이 있으시면 덤으로 말씀해 주셔도 감사하지만, 답을 기대하고 드리는 질문은 아닙니다.

표 8 - 다른 곳에 물을 것

질문	물을 곳	왜 뺐나
개인정보 처리위탁 계약 · 제3자 제공 동의의 요건	법무 자문	계약 문서 작성은 개발 판단이 아님
대학 정보보호 심의 통과 요건 · 인증 필요 여부	학교 정보화 부서 · 보안 컨설팅	학교마다 기준이 다름
학교 예산·발주 시기, 협약서 문안	학교 장학팀 · 산학협력단	행정 절차
메일 접수를 ‘정식 접수’로 인정할지	학교 장학팀	학교의 정책 결정

다만 Q1·Q3에는 “학교가 보통 요구하는 기술 조건”이 포함돼 있습니다. 이는 심의 통과 여부가 아니라 **구현에 무엇이 필요한지**를 여쭙는 것이라 남겨 두었습니다.

C. 저장소 구조

```

hanggonggan/
├─ index.html · app.js · style.css · sw.js      앱 본체 (빌드 없음)
├─ match-engine.js · parse-*.js · section-head.js  판정 엔진 (화면·워커 공용)
├─ forms.js · form-plan.js · essay*.js          양식·서술형 문서
├─ supabase-client.js · notify*.js · chat.js      로그인·알림·도우미
├─ data/
│   ├── registered.json      정식 등록 공고 72건 · 199 KB
│   ├── forms.json          양식 스키마 48종 · 178 KB
│   ├── notices.json       실시간 공고 57건 · 34 KB
│   ├── kosaf-open.json     한국장학재단 재단 101곳 · 202 KB
│   └─ tuition.json majors.json search-index.json
├─ collector/              수집 로봇 47개 (.mjs) + 원문 542개 파일
├─ server/
│   ├── push/              푸시 발송 - 가동 중
│   ├── mail-worker.js     접수 메일 - 미배포
│   └─ essay/ chat/        AI 초안·도우미 - 미배포
├─ verify/                 검증 드라이버 42개 (브라우저 13 + Node)
├─ _admin/                 관리자 화면 (Cloudflare Pages + Access)
└─ .github/workflows/      자동화 27종

```

D. 용어

정식 등록	사람이 검수해 매칭·양식 작성까지 붙인 공고. 로봇이 자동 등록한 것은 ‘검수 전’ 배지가 붙습니다.
발체	공고 원문에서 그대로 잘라 낸 문장. 앱은 신청 방법을 추론하지 않고 발체만 보여 줍니다.
반클리	학교 허락 없이 가능한 만큼 — 정확한 화면으로 데려다주고 넣을 값을 미리 갖춰 두는 것.
감사 관문	데이터를 저장하기 직전에 전수 검사해, 실패하면 그 실행의 변경을 되돌리는 장치.
양식	공고에 첨부된 실제 신청서 파일과 같은 구조 의 문서. 자유 서식으로 대체하지 않습니다.

E. 확인하실 수 있는 것

- 앱 — seonju5543-web.github.io/hanggonggan (설치 없이 브라우저에서 바로 동작)
- 저장소 — github.com/seonju5543-web/hanggonggan (공개 · 이 문서의 모든 코드 인용은 여기서 그대로 가져왔습니다)
- 원문 조사 — 121건 채널 분류와 포털형 18건의 근거 문장은 별도 문서로 제공 가능합니다.
- 필요하시면 테스트 계정과 데모 데이터를 준비해 드리겠습니다.

한대장 · 기술 고문 요청서 · 2026-09-23 고침 · 이선주 seonju5543@gmail.com

이 문서의 수치는 전부 **2026-09-23** 저장소 실측이며, 채널 분류는 2026-09-02 게시판 직접 열람 결과입니다. 수집 대상은 **경희대 · 한국외대 2곳**입니다(2026-09-21 결정).